

E.R.I.K - Human Resource & Payroll

Summary / Scope

ERIK's HR & Payroll subsystem must support the full employee lifecycle and payroll operations for multi-company, multi-branch clients. It must be modular (individual modules can be enabled per client), Smart-Invoice/OCR aware where relevant, support TPIN/tax validation, multi-currency payroll, role-based access, audit trails, export to accounting, and both a Python frontend (Streamlit / Replit-ready) and a FastAPI backend. Everything should be secure, auditable, and configurable to local compliance (Zambia) with the ability to adapt to other jurisdictions.

High-level modules (priority + short description)

Use these as separate code modules/services (microservices or modules within a monolith).

1. **Job Requisition & Approval Workflow**
 - Create, approve, track job requisitions; link to budgets and positions.
2. **User Role & Permission Matrix Manager**
 - Admin UI for roles, permissions, scopes (company/branch/department), and enforced role checks.
3. **Audit Trail Viewer & Access Control Logs**
 - Immutable logs for all HR/payroll actions. Exportable, searchable by user / date / action.
4. **Promotion, Demotion & Transfer Tracker**
 - Track personnel moves, effective dates, approvals, and salary adjustments.
5. **Employment Contract Management**
 - Store contract templates, signed contracts (file storage), auto-generate contract letters.
6. **HR Letters & Document Generator**
 - Generate appointment, confirmation, termination, NOC, salary verification letters (PDF/Word).
7. **HR Analytics & Workforce Insights Dashboard**
 - Headcount, turnover rate, FTE, leave balances, gender split, cost by department.
8. **HR Budgeting & Forecasting Engine**
 - Department-level salary budget planning, headcount plans, salary increase modeling.
9. **HR Cost Allocation & Departmental Spend Tracker**
 - Allocate payroll costs to departments/projects; integrate to GL.
10. **(User requested) HR & Payroll Module**
 - The primary payroll engine: payslips, taxes, statutory deductions, salary advances, loans.
11. **Loans & Salary Advances Management**

- Requests, approval, amortization schedules, integration to payroll for deductions.
- 12. **Skills & Competency Tracker**
 - Employee skill records, training history, certification expiries.
- 13. **(Other HR operations)**
 - Onboarding/offboarding automation, background checks, equipment assignment.
- 14. (Procurement/Smart Invoice/OCR)
 - Integrations where payroll interacts with procurement (e.g., petty cash, staff purchases, vendor payments) or when OCR affects invoices for contractor payments.
- 15. **Petty Cash Request & Float Management**
 - Staff petty cash requests (for payroll/HR petty expenses), approval and reconciliation.

Non-functional requirements

- **Backend:** FastAPI (Python) — consistent with project context. Use Pydantic models, OpenAPI docs, and JWT + RBAC middleware.
- **Frontend:** Two formats for each feature:
 - HTML/CSS/JS (vanilla or React) for public web UI.
 - Python frontends (Streamlit) for internal admin tools / demos — deployable on Replit/Render/Heroku/Vercel.
- **DB:** PostgreSQL (production) with support for JSONB for flexible configs; optionally use Redis for caching. Migrations via Alembic.
- **File storage:** S3-compatible (minio) or cloud provider; store signed contracts, payslips (PDF) and scanned IDs.
- **Security:** TLS, password hashing (bcrypt/argon2), rate limiting, audit logs, encryption-at-rest for sensitive fields (IDs, bank details).
- **Internationalization:** Multi-currency support; date/time localized to Africa/Lusaka by default.
- **Testing:** Unit, integration, and end-to-end tests. CI pipeline (GitHub Actions).
- **Observability:** Structured logs, metrics, and error tracking (Sentry).

Data model (core tables / models)

I'll present simplified SQL-like schemas. Expand with indexes and constraints.

1. **companies:** id, name, registration_no, default_currency, address, contact, tax_tpin, config_json
2. **branches:** id, company_id, name, address, timezone
3. **departments:** id, company_id, name, manager_user_id
4. **roles:** id, name, permissions (JSON)
5. **users:** id, email, hashed_password, full_name, roles (JSON), company_id, is_active, last_login

6. **employees:** id, company_id, user_id (nullable), employee_no, first_name, last_name, id_number, dob, gender, nationality, date_joined, contract_type, position, department_id, supervisor_id, employment_status, bank_account_json, tax_tpin, home_currency
7. **contracts:** id, employee_id, template_id, start_date, end_date, salary_base, signed_file_path, status
8. **positions:** id, title, grade, default_salary_band_min, max, job_description
9. **job_requisitions:** id, company_id, department_id, position_id, created_by, approvals_json, status
10. **leave_requests:** id, employee_id, leave_type, start_date, end_date, days, status, approval_flow_json
11. **holidays:** id, company_id, date, description, local_flag
12. **skills:** id, name, description
13. **employee_skills:** id, employee_id, skill_id, level, expiry_date
14. **loans:** id, employee_id, amount, outstanding_balance, interest_rate, repayment_schedule_json, status
15. **salary_components:** id, company_id, name, type (earnings/deduction), taxable_bool, calculation_formula (JSON)
16. **payruns:** id, company_id, period_start, period_end, currency, status, total_gross, total_net, created_by, posted_to_gl_bool
17. **payslips:** id, payrun_id, employee_id, gross, deductions_json, net, pdf_path
18. **tax_settings:** id, company_id, jurisdiction, tax_brackets_json, employer_contributions_json
19. **audit_logs:** id, user_id, action, entity_type, entity_id, timestamp, details_json, ip_address
20. **access_logs:** id, user_id, action, timestamp, metadata_json

Payroll logic & calculations (high level)

- **Salary elements:** Base salary + allowances + benefits (taxable or non-taxable) - deductions (PAYE/similar, social security, loans, advances, pension).
- **Configurable salary components** defined per company; formula engine supports:
 - Fixed amounts
 - Percent of base
 - Conditional formulas (e.g., overtime = hours * rate)
 - JSON representation and a safe formula evaluator
- **Tax engine:** modular tax plug-ins per jurisdiction. For Zambia: TPIN validation (see integrations). Tax brackets defined via `tax_settings`. Provide employer/employee split for statutory contributions.
- **Multi-currency:** calculate components in employee home currency. Store exchange rate used per payrun (source: company rates or manual). Round handling configurable.
- **Salary advances & loans:** amortization schedules; integrate with payroll to deduct agreed amounts; track outstanding balance and interest.
- **Payslip generation:** HTML -> PDF via wkhtmltopdf or WeasyPrint; store in file storage and email to employee.
- **Payrun process:** Draft -> Validate (policy checks: approvals, bank details present) -> Post -> Export to bank/GL -> Archive.

- **Edge cases:** Retroactive pays (back pay), off-cycle payments, terminations and final settlements, statutory leave pay.

API design (example endpoints — use REST + OpenAPI)

All endpoints require JWT and role check.

Auth:

- POST /api/v1/auth/login
- POST /api/v1/auth/refresh
- POST /api/v1/auth/logout

Companies / Config:

- GET /api/v1/companies/{id}
- PATCH /api/v1/companies/{id}/config

Employees:

- POST /api/v1/employees
- GET /api/v1/employees/{id}
- PUT /api/v1/employees/{id}
- GET /api/v1/employees?company={}&department={} (filters)
- POST /api/v1/employees/{id}/documents (contracts, IDs)
- GET /api/v1/employees/{id}/payslips

Payroll / Payruns:

- POST /api/v1/payruns (create draft)
- GET /api/v1/payruns/{id}
- POST /api/v1/payruns/{id}/validate
- POST /api/v1/payruns/{id}/post
- POST /api/v1/payruns/{id}/export (CSV / bank / GL)
- GET /api/v1/payruns/{id}/payslips
- POST /api/v1/payslips/{id}/email

Salary components:

- GET /api/v1/salary-components
- POST /api/v1/salary-components

Loans & Advances:

- POST /api/v1/loans
- GET /api/v1/loans/{id}
- POST /api/v1/loans/{id}/repay

Leaves:

- POST /api/v1/leaves
- GET /api/v1/leaves/{id}
- POST /api/v1/leaves/{id}/approve

Reporting:

- GET /api/v1/reports/headcount?from=&to=&group_by=
- GET /api/v1/reports/payroll-summary?period=
- GET /api/v1/reports/cost-allocation?period=&department=

Audit:

- GET /api/v1/audit?entity=&user=&from=&to=

RBAC & Permissions

- Role models: SuperAdmin, CompanyAdmin, HRManager, PayrollOfficer, DepartmentManager, Employee, Auditor
- Permissions granular (CRUD on employees, run payroll, approve requisitions, export GL, view payslips)
- UI must show only allowed actions based on role and scope (company/branch/department).

UI / UX requirements (per module)

Provide both a Streamlit Python admin UI and HTML/CSS/JS mockups.

Key screens:

- Employee Directory (search, filters, badges for status)
- Employee profile (tabs: Personal, Job, Payroll, Contracts, Documents, Skills, Loans)
- Payslip Viewer (PDF inline), download, email
- Payrun wizard: configuration (period, exchange rates), run preview, validation issues, approval flow
- Approvals dashboard (job requisitions, leaves, loans)
- Reports dashboard (tiles + export)
- Audit log viewer with filters and export
- Role & Permission editor (matrix UI)

- Onboarding wizard (automatic checklist and tasks)

Mockups must include:

- Clear action buttons: Draft, Validate, Post, Export
- Inline warnings/errors (e.g., missing bank details)
- Mobile-responsive layout.

Integrations

- **Bank file export:** formats for local banks (CSV/TXT); generic SEPA/ISO20022 optional.
- **Accounting/GL:** post journal entries via API (interface to ERP accounting module).
- **Tax authority / TPIN validation:** integrate an endpoint to validate TPIN numbers (store validity result + last-checked timestamp). If local API not available, provide uploader or manual verification flag.
- **Email/SMS:** for payslip distribution & alerts.
- **OCR / Smart Invoice:** integrate for contractor invoices, reimbursement claims; use OCR modules for attachments (Module 79).
- **SSO / LDAP:** optional for enterprise customers.

Compliance & statutory features (Zambia-focused, but configurable)

- TPIN validation & storage
- PAYE calculation by bracket
- Employer statutory contributions (Napsa/NSS, Pension) — configurable
- Leave accrual rules, maternity/paternity handling, termination procedures & final settlements
- Export formats for statutory reporting
- Data retention policy, right to delete (subject to legal constraints)

Notifications & workflows

- Approval flows for job requisitions, leave, loans, salary changes
- Webhooks for external systems (accounting, bank)
- Email templates (configurable) for onboarding, payslip, salary change, tax certificates

Audit, Logging, & Security

- Immutable `audit_logs` for every state-changing action

- `access_logs` for authentication events
- Data encryption: bank account numbers, national IDs
- Pen tests and OWASP baseline
- Role and permission change logs

Acceptance criteria (per feature)

Example: Payrun

- Create payrun draft → system calculates gross/net for at least 10 employees with mixed components
- Validation step returns missing bank details for employees without bank account
- Post payrun creates payslips (PDF) and journal lines for GL export
- Exported bank file matches the specified CSV template
- Full audit entries for creation, validation, posting, export

Example: Employee onboarding

- Creating employee auto-generates checklist (ID upload, bank details, tax TPIN)
- Employee receives email invite with link to view payslips and documents

Tests & QA

- Unit tests for tax calculation engine and formula evaluator
- Integration test for payrun → payslip PDF generation → email sending
- E2E tests for user flows (onboarding, leave approval, loan deduction)
- Load test: ensure payrun for 5000 employees within acceptable time (define target e.g., complete run in X minutes — pick later with infra team)

Implementation roadmap (sprint-friendly)

I'll suggest a roadmap the dev team can tailor — sprint durations depending on team size.

Sprint 0 (Setup)

- Repo scaffold, CI/CD, infra (DB, file storage), auth & RBAC base
- Base API and user model
- Role & Permission Manager UI

Sprint 1

- Employee model CRUD + documents
- Employee profile UI (Streamlit + HTML)

- Basic audit logs

Sprint 2

- Salary components & company config
- Tax settings model (stub for Zambia)
- Payslip HTML -> PDF pipeline

Sprint 3

- Payrun engine (draft, preview)
- Validation & approval flows
- Payslip generation + email

Sprint 4

- Loans & advances (Module 19)
- Leave module & holiday calendar
- Export to GL & bank file

Sprint 5

- Reporting & analytics (Module 52)
- Budgeting & forecasting (Module 54)
- Integrations (TPIN validation endpoint, OCR as needed)

Sprint 6+

- Final cleanups, mobile responsiveness, performance tuning, hardened security

Deliverables for developers (checklist)

- FastAPI project with OpenAPI documentation and examples.
- PostgreSQL schema + Alembic migrations for HR & payroll models listed above.
- Streamlit UI pages for each major user flow (employee, payrun, approvals, reports).
- HTML/CSS/JS front-end equivalents (component-based if React).
- Unit and integration tests with CI configured.
- Sample data seeders for testing (companies, employees, tax settings).
- Export templates for bank and GL (sample CSV/JSON).
- README with deployment steps, env variables, and architecture diagram.
- Postman collection or equivalent API examples.
- Security & data protection checklist.

Example: Simple DB schema snippet (Postgres)

```
CREATE TABLE employees (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  company_id UUID NOT NULL REFERENCES companies(id),  
  employee_no TEXT UNIQUE,  
  first_name TEXT NOT NULL,  
  last_name TEXT NOT NULL,  
  id_number TEXT,  
  dob DATE,  
  date_joined DATE,  
  department_id UUID,  
  position_id UUID,  
  tax_tpin TEXT,  
  bank_account JSONB,  
  home_currency TEXT,  
  status TEXT,  
  created_at TIMESTAMP DEFAULT now(),  
  updated_at TIMESTAMP DEFAULT now()  
);
```

Example: Payrun endpoint (pseudocode)

POST /api/v1/payruns

Request:

```
{  
  "company_id": "uuid",  
  "period_start": "2025-10-01",  
  "period_end": "2025-10-31",  
  "currency": "ZMW",  
  "exchange_rate": 1.0,  
  "employee_filters": {"department": "sales"}  
}
```

Response: payrun draft id + summary of counts + preview URL

Developer notes / gotchas

- **Formula evaluator:** do not use `eval` on raw input. Use a safe expression evaluator (asteval or build AST with limited functions).
- **Rounding:** floating-point rounding must be predictable (use decimal libraries).
- **Back pay and retro adjustments:** design payrun with adjustments and retro-pays as separate line items.
- **Privacy:** obey data retention and allow export/deletion workflows; but ensure payroll legal retention is considered.

- **Performance:** bulk payrun computations should be batched and use worker queues (Celery) for large client runs.
- **Migration path:** ensure ability to import legacy employee/payroll data via CSV templates.

Example acceptance tests (short)

1. Create employee with tax_tpin and bank account → create payrun for employee → generated payslip PDF with gross/deductions/net present.
2. Loan created with schedule → payrun applies deduction → loan outstanding reduced accordingly.
3. Remove role permission for “PayrollOfficer” → that user cannot trigger payrun posting.

Implementation examples & code style expectations

- Pydantic models for API schema.
- Clear service layer (business logic separated from controllers).
- Use repositories for DB access to ease testing.
- Logging format JSON, include request_id.

Next actions for the developer

1. Clone starter repo (create one if missing) and implement Sprint 0 items.
2. Run DB migrations and seed sample data.
3. Implement employee CRUD & profile UI.
4. Implement salary component and preliminary tax engine stub.
5. Schedule a demo after Sprint 3 with sample payrun for feedback.